

Министерство образования Республики Беларусь
Учреждение образования
«Белорусский государственный университет информатики и радиоэлектроники»
Филиал
«Минский радиотехнический колледж»

Учебный предмет
«Разработка кроссплатформенных приложений»

Отчёт
по выполнению лабораторной работы
«Разработка, отладка и испытание программ с потоками различных
приоритетов»

Выполнил: учащийся гр. 3к9391
Шутько А.П.
Проверил: Грунтович П.Н.

Минск 2025

Лабораторная работа № 15

Тема работы: «Разработка, отладка и испытание программ с потоками различных приоритетов»

1. Цель работы

Обучить управлению потоками посредством изменения приоритетов потоков.

2. Задание

Номер варианта соответствует вашему номеру по списку, если номер по списку больше последнего варианта, то необходимо начать подсчет варианта с начала (например, учащийся с номером по списку 22 должен выполнить 2 вариант).

3. Оснащение работы

Задание по варианту, ЭВМ, среда разработки.

4. Выполнения работы

Задание 1: В задании лабораторной работы № 14 необходимо создать несколько объектов классов, для каждого из них задать различный приоритет, запустить все потоки одновременно, и проанализировать, как приоритет потока влияет на производительность:

```
import java.util.Locale;

class VigenereCipher extends Thread {
    private final String key;
    private final String data;
    private final boolean encryptMode;
    private String result;
    private long executionTime;

    public VigenereCipher(String key, String data, boolean encryptMode) {
        this.key = key.toUpperCase();
        this.data = data;
        this.encryptMode = encryptMode;
    }

    private String encrypt(String text, String key) {
        StringBuilder res = new StringBuilder();
        text = text.toUpperCase();
        for (int i = 0, j = 0; i < text.length(); i++) {
```

```

        char c = text.charAt(i);
        if (c < 'A' || c > 'Z') continue;
        res.append((char) ((c + key.charAt(j) - 2 * 'A') % 26 + 'A'));
        j = (j + 1) % key.length();
    }
    return res.toString();
}

private String decrypt(String text, String key) {
    StringBuilder res = new StringBuilder();
    text = text.toUpperCase();
    for (int i = 0, j = 0; i < text.length(); i++) {
        char c = text.charAt(i);
        if (c < 'A' || c > 'Z') continue;
        res.append((char) ((c - key.charAt(j) + 26) % 26 + 'A'));
        j = (j + 1) % key.length();
    }
    return res.toString();
}

@Override
public void run() {
    long startTime = System.nanoTime();

    if (encryptMode) {
        for (int i = 0; i < 1000000; i++) {
            Math.sqrt(i);
        }
        this.result = encrypt(data, key);
        System.out.println(Thread.currentThread().getName() + " [ENCRYPT]: "
+ result);
    } else {
        for (int i = 0; i < 1000000; i++) {
            Math.sqrt(i);
        }
        this.result = decrypt(data, key);
        System.out.println(Thread.currentThread().getName() + " [DECRYPT]: "
+ result);
    }

    executionTime = System.nanoTime() - startTime;
    System.out.println(Thread.currentThread().getName() + " [ПРИОРИТЕТ=" +
getPriority() +
        "]" + " время выполнения: " + (executionTime / 1_000_000) + " мс");
}

public String getResult() {
    return result;
}

public long getExecutionTime() {
    return executionTime;
}
}

public class Main {
    public static void main(String[] args) {
        String originalText = "HELLO WORLD THIS IS A LONGER TEXT FOR BETTER
ANALYSIS";
    }
}

```

```

        String keyForAlphaBeta = "KEY";

        VigenereCipher task1 = new VigenereCipher(keyForAlphaBeta, originalText,
true);
        VigenereCipher task2 = new VigenereCipher(keyForAlphaBeta, "RIJVSUYVJN",
false);
        VigenereCipher task3 = new VigenereCipher("JAVA", "CONCURRENCY IS COOL
AND THIS IS ADDITIONAL TEXT FOR THREAD GAMMA", true);

        task1.setName("Thread-Alpha");
        task2.setName("Thread-Beta");
        task3.setName("Thread-Gamma");

        task1.setPriority(Thread.MIN_PRIORITY);
        task2.setPriority(Thread.NORM_PRIORITY);
        task3.setPriority(Thread.MAX_PRIORITY);

        System.out.println("=== ЗАПУСК ПОТОКОВ С РАЗНЫМИ ПРИОРИТЕТАМИ ===\n");
        System.out.println("Thread-Alpha: приоритет " + task1.getPriority() + "
(MIN)");
        System.out.println("Thread-Beta: приоритет " + task2.getPriority() + "
(NORM)");
        System.out.println("Thread-Gamma: приоритет " + task3.getPriority() + "
(MAX)\n");

        long overallStartTime = System.nanoTime();

        // Запуск всех потоков
        task1.start();
        task2.start();
        task3.start();

        try {
            task1.join();
            task2.join();
            task3.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }

        long overallEndTime = System.nanoTime();

        System.out.println("\n=== АНАЛИЗ ПРОИЗВОДИТЕЛЬНОСТИ ===");
        System.out.println("Общее время выполнения: " + ((overallEndTime -
overallStartTime) / 1_000_000) + " мс\n");

        System.out.println("Статистика по потокам:");
        System.out.printf("%-15s | %-10s | %-20s\n", "Поток", "Приоритет",
"Время выполнения (мс)");
        System.out.println("-----");
        System.out.printf("%-15s | %-10d | %-20d\n", task1.getName(),
task1.getPriority(), task1.getExecutionTime() / 1_000_000);
        System.out.printf("%-15s | %-10d | %-20d\n", task2.getName(),
task2.getPriority(), task2.getExecutionTime() / 1_000_000);
        System.out.printf("%-15s | %-10d | %-20d\n", task3.getName(),
task3.getPriority(), task3.getExecutionTime() / 1_000_000);

        System.out.println("\n=== АНАЛИЗ ВЛИЯНИЯ ПРИОРИТЕТОВ ===");
        System.out.println("1. Thread-Gamma (приоритет 10 - MAX):");

```

```

        System.out.println("    - Получил наибольшее количество процессорного
времени");
        System.out.println("    - Обычно завершается первым или вторым");
        System.out.println("    - Высокий приоритет дает преимущество в
планировании");

        System.out.println("\n2. Thread-Beta (приоритет 5 - NORM):");
        System.out.println("    - Стандартный приоритет");
        System.out.println("    - Выполняется в обычном режиме");
        System.out.println("    - Время выполнения зависит от загрузки CPU");

        System.out.println("\n3. Thread-Alpha (приоритет 1 - MIN):");
        System.out.println("    - Наименьший приоритет");
        System.out.println("    - Получает CPU только когда нет потоков с более
высоким приоритетом");
        System.out.println("    - Обычно завершается последним");

        System.out.println("\n--- Проверка завершена ---");

        System.out.println("\n=== РЕЗУЛЬТАТЫ РАБОТЫ ===");
        System.out.println("Thread-Alpha (ENCRYPT): \"" + originalText + "\" ->
\"" + task1.getResult() + "\"");
        System.out.println("Thread-Beta (DECRYPT): \"RIJVSUYVJN\" -> \"" +
task2.getResult() + "\"");
        System.out.println("Thread-Gamma (ENCRYPT): \"CONCURRENCY IS COOL...\" -
> \"" + task3.getResult() + "\"");
    }
}

```

```

=== ЗАПУСК ПОТОКОВ С РАЗНЫМИ ПРИОРИТЕТАМИ ===

Thread-Alpha: приоритет 1 (MIN)
Thread-Beta: приоритет 5 (NORM)
Thread-Gamma: приоритет 10 (MAX)

Thread-Beta [DECRYPT]: HELLOWORLD
Thread-Gamma [ENCRYPT]: LOICDRMEWCTIBCJOUAIDCHDSRSVDMIOIXNVLCSTO0MTQRZAMGVMVA
Thread-Alpha [ENCRYPT]: RIJVSUYVJNXFSWGCEJYREOVROBRPSPLIRDIPKRYVCQSW
Thread-Beta [ПРИОРИТЕТ=5] время выполнения: 12 мс
Thread-Gamma [ПРИОРИТЕТ=10] время выполнения: 13 мс
Thread-Alpha [ПРИОРИТЕТ=1] время выполнения: 14 мс

=== АНАЛИЗ ПРОИЗВОДИТЕЛЬНОСТИ ===
Общее время выполнения: 22 мс

Статистика по потокам:
Поток          | Приоритет  | Время выполнения (мс)
-----
Thread-Alpha   | 1          | 14
Thread-Beta    | 5          | 12
Thread-Gamma   | 10         | 13

=== АНАЛИЗ ВЛИЯНИЯ ПРИОРИТЕТОВ ===
1. Thread-Gamma (приоритет 10 - MAX):
  - Получил наибольшее количество процессорного времени
  - Обычно завершается первым или вторым
  - Высокий приоритет дает преимущество в планировании

```

Рисунок 1 – Результат работы программы

```

2. Thread-Beta (приоритет 5 - NORM):
  - Стандартный приоритет
  - Выполняется в обычном режиме
  - Время выполнения зависит от загрузки CPU

3. Thread-Alpha (приоритет 1 - MIN):
  - Наименьший приоритет
  - Получает CPU только когда нет потоков с более высоким приоритетом
  - Обычно завершается последним

--- Проверка завершена ---

=== РЕЗУЛЬТАТЫ РАБОТЫ ===
Thread-Alpha (ENCRYPT): "HELLO WORLD THIS IS A LONGER TEXT FOR BETTER ANALYSIS" -> "RIJVSUYVJNXFSWGCEJYREOVROBRPSPLIRDIPKRYVCQSW"
Thread-Beta (DECRYPT): "RIJVSUYVJN" -> "HELLOWORLD"
Thread-Gamma (ENCRYPT): "CONCURRENCY IS COOL..." -> "LOICDRMEWCTIBCJOUAIDCHDSRSVDMIOIXNVLCSTO0MTQRZAMGVMVA"

```

Рисунок 2 – Результат работы программы

5. Контрольные вопросы и задания

1. Какой приоритет имеет поток по умолчанию?

По умолчанию любой вновь созданный поток в Java наследует приоритет того потока, который его создал. Для главного потока (main thread), который запускает приложение, приоритет по умолчанию устанавливается равным Thread.NORM_PRIORITY, что соответствует числовому значению 5. Следовательно, все потоки, созданные из главного потока без явного указания приоритета, также будут иметь приоритет 5. Если же поток создается из другого потока, который имеет ненормативный приоритет, то новый поток унаследует именно этот приоритет родительского потока. Таким образом, стандартное значение приоритета для большинства пользовательских потоков составляет 5 из диапазона от 1 до 10.

2. Какие можно задать приоритеты для потока?

В Java приоритеты потоков задаются целыми числами в диапазоне от 1 до 10 включительно, где 1 – это минимальный приоритет (Thread.MIN_PRIORITY), 5 – нормальный приоритет (Thread.NORM_PRIORITY), а 10 – максимальный приоритет (Thread.MAX_PRIORITY). Для установки приоритета используется метод setPriority(int newPriority), который принимает целое число или одну из трех констант-полей класса Thread. Важно отметить, что хотя Java предоставляет 10 уровней приоритета, большинство операционных систем поддерживают меньшее количество уровней планирования, поэтому происходит отображение Java-приоритетов на системные уровни. Например, в Windows используется 7 уровней приоритетов, а в некоторых системах Linux может использоваться всего 1 уровень, что делает приоритеты Java лишь рекомендацией для планировщика операционной системы.

3. Как влияет на работу потока изменение приоритета потока?

Изменение приоритета потока влияет на планирование выполнения потоков, определяя, какое количество процессорного времени будет выделено потоку относительно других конкурирующих потоков. Потоки с более высоким приоритетом получают преимущество при выборе планировщиком следующего потока для выполнения, что означает, что они будут выполняться чаще и, как правило, завершаться быстрее, особенно в условиях высокой загрузки процессора и конкуренции за ресурсы. Потоки с низким приоритетом выполняются реже и могут испытывать задержки, уступая процессорное время потокам с более высоким приоритетом. Однако фактическое влияние приоритетов зависит от операционной системы и ее алгоритма планирования: в некоторых системах разница между приоритетами может быть незначительной, в других – более выраженной. Важно понимать, что приоритеты не гарантируют строгого порядка выполнения и не предотвращают ситуаций инверсии приоритетов, а служат лишь механизмом для указания относительной важности потоков планировщику.